

```
1  =====
2  GHAR -- MASTER PROJECT RULES & NOTES
3  =====
4
5
6  1. SINGLE SOURCE OF TRUTH
7  =====
8
9  Every important concept must have one canonical name.
10
11 Example:
12
13 propertyType
14 NOT:
15 type
16 property_type
17 property-type
18
19 userId
20 NOT:
21 user_id
22 userid
23 userID
24
25 createdAt
26 NOT:
27 created_at
28 created
29 dateCreated
30
31
32 The same naming convention must be used across:
33
34 HTML
35 CSS
36 JavaScript
37 JSON
38 API
39 Database
40 Backend
41 Frontend
42
43
44 =====
45 2. NEVER DUPLICATE BUSINESS LOGIC
46 =====
```

```
47
48 A rule should ideally exist in one authoritative place.
49
50 Example:
51
52 Property price validation
53     ↓
54 Backend validation
55
56 Frontend may show the same validation for user experience,
57 but the backend remains authoritative.
58
59
60 =====
61 3. DATABASE IS THE PRODUCTION SOURCE OF TRUTH
62 =====
63
64 JSON:
65 Reference/development/seed data
66
67 PostgreSQL:
68 Production users
69 Production properties
70 Production enquiries
71 Production offers
72 Production bookings
73 Production payments
74 Production documents
75 Production messages
76 Production notifications
77
78
79 Never depend on JSON files for critical production
80 transactions.
81
82
83 =====
84 4. API IS THE BRIDGE
85 =====
86
87 Frontend should not directly connect to PostgreSQL.
88
89 Correct:
90
91 HTML
92     ↓
```

93 JavaScript
94 ↓
95 API
96 ↓
97 Backend
98 ↓
99 Database

100
101
102 Incorrect:

103
104 HTML
105 ↓
106 JavaScript
107 ↓
108 PostgreSQL

109
110
111 =====
112 5. NEVER TRUST THE FRONTEND
113 =====

114
115 Anything important must be verified by the backend.

116
117 Examples:

118
119 Price
120 User role
121 Property ownership
122 Payment status
123 Offer status
124 Booking status
125 Document status
126 Admin permissions
127 Referral rewards

128
129
130 The frontend is **for** interaction.

131
132 The backend is authoritative.

133
134
135 =====
136 6. USER ROLES MUST BE CENTRALIZED
137 =====

138

```
139 Define roles once.
140
141 Example:
142
143 BUYER
144 SELLER
145 AGENT
146 ADMIN
147 SUPER_ADMIN
148
149
150 Do not manually create separate authentication systems
151 for each role.
152
153
154 Use:
155
156 Authentication
157 +
158 Authorization
159 +
160 Permissions
161
162
163 =====
164 7. PROPERTY OWNERSHIP MUST BE VERIFIED
165 =====
166
167 Before modifying a property:
168
169 User
170 ↓
171 Authentication
172 ↓
173 Property ID
174 ↓
175 Ownership check
176 ↓
177 Permission check
178 ↓
179 Modification
180
181
182 A seller must never be able to edit another seller's property
183 by simply changing the property ID in an API request.
184
```

```
185
186 =====
187 8. PRIVATE DATA MUST STAY PRIVATE
188 =====
189
190 Never expose unnecessarily:
191
192 Private phone numbers
193 Private email addresses
194 Identity documents
195 Bank information
196 Passwords
197 Authentication tokens
198 Private property documents
199 Internal admin information
200
201
202 Public property page should expose only the information
203 GHAR intentionally chooses to publish.
204
205
206 =====
207 9. NO SECRETS IN FRONTEND
208 =====
209
210 Never put secret credentials inside:
211
212 HTML
213 CSS
214 JavaScript
215 JSON
216 public GitHub repository
217
218
219 Examples:
220
221 DATABASE_URL
222 JWT_SECRET
223 SESSION_SECRET
224 PAYMENT_SECRET
225 AI_API_KEY
226 SMTP_PASSWORD
227
228
229 These belong in server-side environment variables.
230
```

```
231
232 =====
233 10. ENVIRONMENT SEPARATION
234 =====
235
236 Use separate environments:
237
238 DEVELOPMENT
239 STAGING
240 PRODUCTION
241
242
243 Example:
244
245 Development:
246 local database
247
248 Staging:
249 test database
250
251 Production:
252 production database
253
254
255 Never test experimental database changes directly on
256 production.
257
258
259 =====
260 11. USE DATABASE MIGRATIONS
261 =====
262
263 Do not repeatedly modify production tables manually.
264
265 Use:
266
267 migration 001
268 migration 002
269 migration 003
270 ...
271
272
273 Every database structure change should be reproducible.
274
275
276 =====
```

```
277 12. USE SEED DATA
278 =====
279
280 Create controlled development data.
281
282 Example:
283
284 GHAR-P001
285 GHAR-P002
286 GHAR-P003
287
288
289 Use realistic but fake information.
290
291 Never use real people's sensitive information for testing.
292
293
294 =====
295 13. USE IDs CONSISTENTLY
296 =====
297
298 Every major entity needs a unique ID.
299
300 Examples:
301
302 USR-000001
303 GHAR-P000001
304 ENQ-000001
305 OFR-000001
306 VIS-000001
307 BKG-000001
308 PAY-000001
309 DOC-000001
310 MSG-000001
311
312
313 The database may use UUIDs internally if desired.
314
315 The public/business identifier can follow the GHAR format.
316
317
318 =====
319 14. USE STATUS MACHINES
320 =====
321
322 Important objects should have controlled statuses.
```

323
324 PROPERTY:
325
326 draft
327 pending
328 approved
329 rejected
330 active
331 paused
332 sold
333 rented
334 expired
335 archived
336
337
338 OFFER:
339
340 draft
341 submitted
342 countered
343 accepted
344 rejected
345 expired
346 cancelled
347 completed
348
349
350 BOOKING:
351
352 pending
353 confirmed
354 cancelled
355 completed
356 expired
357
358
359 PAYMENT:
360
361 pending
362 processing
363 successful
364 failed
365 cancelled
366 refunded
367
368

```
369 Do not allow random status strings.
370
371
372 =====
373 15. AUDIT IMPORTANT ACTIONS
374 =====
375
376 Keep audit logs for important events.
377
378 Examples:
379
380 login
381 logout
382 property_created
383 property_updated
384 property_approved
385 property_rejected
386 offer_created
387 offer_accepted
388 booking_created
389 payment_success
390 document_verified
391 admin_action
392 permission_change
393
394
395 Audit logs should contain enough information to understand
396 what happened and when.
397
398
399 =====
400 16. SOFT DELETE WHERE APPROPRIATE
401 =====
402
403 For important records, avoid immediately destroying data.
404
405 Instead consider:
406
407 deletedAt
408 deletedBy
409 deletedReason
410
411
412 Especially for:
413
414 Properties
```

415 Users
416 Offers
417 Bookings
418 Transactions
419 Documents
420
421
422 Financial records should generally have strong retention and
423 audit requirements.
424
425
426 =====
427 17. TRANSACTIONS MUST BE ATOMIC
428 =====
429
430 For critical operations:
431
432 BEGIN
433 ↓
434 perform operation
435 ↓
436 verify everything
437 ↓
438 COMMIT
439
440 If something fails:
441
442 ROLLBACK
443
444
445 Especially important for:
446
447 Payments
448 Bookings
449 Offers
450 Rewards
451 Financial transactions
452
453
454 =====
455 18. NEVER TRUST PAYMENT SUCCESS FROM THE BROWSER
456 =====
457
458 Payment flow:
459
460 Frontend

461 ↓
462 Backend
463 ↓
464 Payment gateway
465 ↓
466 Gateway verification/webhook
467 ↓
468 Backend verification
469 ↓
470 Database
471 ↓
472 Invoice
473
474
475 The browser must never be the final authority on whether
476 money was successfully received.
477
478
479 =====

480 19. FILE STORAGE MUST BE SEPARATE

481 =====

482
483 Do not treat uploaded documents like normal website images.
484
485 Public assets:
486
487 logo
488 icons
489 property marketing images
490 banners
491
492
493 Private assets:
494
495 identity documents
496 ownership documents
497 bank documents
498 agreements
499 verification documents
500
501
502 Private files need controlled access.
503
504
505 =====

506 20. IMAGE SYSTEM

```
507 =====
508
509 Property images should have:
510
511     propertyId
512     imageId
513     storage location
514     sort order
515     alt text
516     status
517     createdAt
518
519
520 Use optimized images.
521
522 Do not load huge original images everywhere.
523
524
525 =====
526 21. PAGINATION EVERYWHERE
527 =====
528
529 Never load thousands of records at once.
530
531 Use:
532
533     page
534     limit
535     offset
536
537 or cursor-based pagination where appropriate.
538
539
540 Apply to:
541
542     Properties
543     Users
544     Enquiries
545     Messages
546     Notifications
547     Payments
548     Admin tables
549     Reports
550
551
552 =====
```

```
553 22. SEARCH MUST BE DESIGNED PROPERLY
554 =====
555
556 Property search should eventually support:
557
558 Location
559 Property type
560 Listing type
561 Price
562 Bedrooms
563 Bathrooms
564 Area
565 Amenities
566 Furnishing
567 Availability
568 Verification
569 Sort
570 Pagination
571
572
573 Search should be handled by the backend/database for
574 production-scale data.
575
576
577 =====
578 23. MOBILE-FIRST DESIGN
579 =====
580
581 Every major feature must work on:
582
583 Mobile
584 Tablet
585 Laptop
586 Desktop
587
588
589 Do not finish desktop first and discover later that the
590 mobile architecture is broken.
591
592
593 =====
594 24. ACCESSIBILITY
595 =====
596
597 Use:
598
```

599 semantic HTML
600 labels
601 keyboard navigation
602 focus states
603 ARIA where appropriate
604 sufficient contrast
605 alt text
606 accessible forms
607
608
609 Images must have meaningful alt text where appropriate.
610
611
612 =====
613 25. SEO
614 =====
615
616 Public property pages should eventually support:
617
618 title
619 description
620 canonical URL
621 Open Graph
622 structured data
623 location information
624 property information
625 sitemap
626 robots.txt
627
628
629 Private dashboards do not need to be treated like public
630 SEO pages.
631
632
633 =====
634 26. PERFORMANCE
635 =====
636
637 Plan for:
638
639 lazy-loaded images
640 compressed images
641 pagination
642 caching
643 database indexes
644 minified production assets

```
645 efficient API responses
646 code splitting where useful
647
648
649 Never sacrifice basic correctness just for a performance
650 optimization.
651
652
653 =====
654 27. ERROR HANDLING
655 =====
656
657 Every API request should have:
658
659 loading state
660 success state
661 empty state
662 error state
663 retry option where appropriate
664
665
666 Never show users:
667
668 SQL errors
669 stack traces
670 secret information
671 internal server paths
672
673
674 Instead show a useful human-readable message.
675
676
677 =====
678 28. LOGGING
679 =====
680
681 Use structured server logs.
682
683 Important:
684
685 timestamp
686 request ID
687 user ID where appropriate
688 endpoint
689 status
690 duration
```

691 **error** code
692
693
694 Never log:
695
696 passwords
697 payment secrets
698 authentication secrets
699 private documents
700 sensitive personal information unnecessarily
701
702
703 =====
704 29. RATE LIMITING
705 =====
706
707 Protect sensitive endpoints.
708
709 Especially:
710
711 login
712 signup
713 OTP
714 password reset
715 AI
716 contact/enquiry
717 messaging
718 payment creation
719
720
721 =====
722 30. API VERSIONING
723 =====
724
725 Consider:
726
727 /api/v1/...
728
729
730 rather than permanently using:
731
732 /api/...
733
734
735 This gives GHAR room to evolve without immediately breaking
736 existing clients.

```
737
738
739 =====
740 31. API DOCUMENTATION
741 =====
742
743 Maintain a master API registry.
744
745 For every endpoint document:
746
747 METHOD
748 URL
749 AUTH REQUIRED
750 ROLE
751 REQUEST
752 RESPONSE
753 ERRORS
754
755
756 Example:
757
758 POST
759 /api/v1/properties
760
761 AUTH:
762 YES
763
764 ROLE:
765 SELLER
766
767 REQUEST:
768 property data
769
770 RESPONSE:
771 created property
772
773
774 =====
775 32. FRONTEND PAGE REGISTRY
776 =====
777
778 Maintain one document listing:
779
780 Page name
781 Filename
782 Purpose
```

```
783 Required role
784 JS file
785 CSS dependency
786 API endpoints
787 Status
788
789
790 Example:
791
792 Seller Dashboard
793
794 File:
795 seller-dashboard.html
796
797 Role:
798 SELLER
799
800 JS:
801 seller-dashboard.js
802
803 APIs:
804 GET /api/v1/seller/dashboard
805
806
807 =====
808 33. JAVASCRIPT MODULE REGISTRY
809 =====
810
811 For every JS file record:
812
813 Filename
814 Purpose
815 Dependencies
816 Pages using it
817 APIs used
818
819
820 This prevents duplicate scripts and circular dependencies.
821
822
823 =====
824 34. CSS CLASS REGISTRY
825 =====
826
827 Use a predictable naming strategy.
828
```

829 Example:

830
831 ghar-header
832 ghar-container
833 ghar-card
834 ghar-property-card
835 ghar-property-grid
836 ghar-btn
837 ghar-btn-primary
838 ghar-modal
839

840
841 Avoid random classes such as:

842
843 box1
844 newcard
845 test2
846 finalcard
847 new-final-card
848

849

850 =====

851 35. DATA ATTRIBUTE REGISTRY

852 =====

853

854 Maintain one master list of dynamic HTML attributes.

855

856 Example:

857

858 data-property-id
859 data-user-id
860 data-seller-stat
861 data-property-action
862 data-modal
863 data-api
864 data-role
865

866

867

868 Do not create similar attributes with different meanings.

869

870

871 =====

871 36. NO "FINAL2", "FINAL3", "NEW", ETC.

872 =====

873

874 Avoid files such as:

```
875
876 style-final.css
877 style-new.css
878 style-final-new.css
879 app2.js
880 dashboard-new.js
881
882
883 Use version control/Git instead.
884
885
886 =====
887 37. GIT RULE
888 =====
889
890 Before major changes:
891
892 COMMIT
893
894
895 Example:
896
897 git commit
898 "Create property search foundation"
899
900
901 Then make the next change.
902
903
904 If something breaks:
905
906 ROLLBACK
907
908
909 Git becomes our safety net.
910
911
912 =====
913 38. FEATURE BRANCHES
914 =====
915
916 For major features:
917
918 main
919 |
920 └─ feature/authentication
```

```
921 | feature/property-engine
922 | feature/buyer-system
923 | feature/seller-system
924 | feature/payments
925 | feature/ai
```

926

927

928 Merge only after testing.

929

930

931 =====

932 39. NEVER CHANGE MANY SYSTEMS AT ONCE

933 =====

934

935 Example:

936

937 Do not simultaneously modify:

938

939 database

940 backend

941 API

942 seller dashboard

943 payment system

944 AI

945

946

947 unless the feature genuinely requires it.

948

949

950 Change the smallest number of components necessary.

951

952

953 =====

954 40. TEST AFTER EVERY PHASE

955 =====

956

957 PHASE COMPLETE

958 ↓

959 TEST

960 ↓

961 FIX

962 ↓

963 COMMIT

964 ↓

965 BACKUP

966 ↓

967 NEXT PHASE
968
969
970 Never carry known critical errors into the next phase.
971
972
973 =====
974 41. BACKUP STRATEGY
975 =====
976
977 Keep:
978
979 Git repository
980 Database backups
981 Environment configuration backup
982 Migration files
983 Important uploaded assets
984
985
986 Never depend on a single copy of the project.
987
988
989 =====
990 42. DOCUMENTATION
991 =====
992
993 Maintain:
994
995 README.md
996 ARCHITECTURE.md
997 API.md
998 DATABASE.md
999 SECURITY.md
1000 DEPLOYMENT.md
1001 CHANGELOG.md
1002
1003
1004 These documents should evolve with the application.
1005
1006
1007 =====
1008 43. CHANGELOG
1009 =====
1010
1011 Every significant change should be recorded.
1012

1013 Example:
1014
1015 Version 0.1.0
1016
1017 Added:
1018 Property search
1019 Property details
1020 Authentication foundation
1021
1022 Fixed:
1023 Mobile navigation
1024 Property card overflow
1025
1026
1027 =====
1028 44. DO NOT OVERBUILD TOO EARLY
1029 =====
1030
1031 Build the reliable core first.
1032
1033 CORE:
1034
1035 Authentication
1036 Properties
1037 Search
1038 Buyer
1039 Seller
1040 Enquiry
1041 Visit
1042 Offer
1043 Booking
1044 Payment
1045 Admin
1046
1047
1048 Then:
1049
1050 AI
1051 Advanced analytics
1052 Rewards
1053 Advanced automation
1054 Advanced recommendation systems
1055
1056
1057 =====
1058 45. AI MUST NOT BECOME THE CORE DATABASE

```
1059 =====
1060
1061 AI is an intelligence layer.
1062
1063 Correct:
1064
1065 User
1066   ↓
1067 AI
1068   ↓
1069 Structured intent
1070   ↓
1071 GHAR API
1072   ↓
1073 Database
1074
1075
1076 Not:
1077
1078 User
1079   ↓
1080 AI
1081   ↓
1082 Direct database modification
1083
1084
1085 =====
1086 46. THIRD-PARTY SERVICES MUST BE ABSTRACTED
1087 =====
1088
1089 For:
1090
1091 Payment
1092 Maps
1093 Email
1094 SMS
1095 AI
1096 Storage
1097
1098
1099 Use a service layer.
1100
1101 Example:
1102
1103 paymentService
1104
```

1105 instead of allowing payment-provider-specific code to spread
1106 throughout every HTML/JS file.
1107
1108
1109 This makes changing providers much easier.
1110
1111
1112 =====
1113 47. DESIGN FOR FAILURE
1114 =====
1115
1116 Assume:
1117
1118 API can fail
1119 database can temporarily fail
1120 payment can fail
1121 email can fail
1122 SMS can fail
1123 AI can fail
1124 image upload can fail
1125 internet can disconnect
1126
1127
1128 GHAR should show useful fallback states instead of breaking.
1129
1130
1131 =====
1132 48. NO DEAD CODE
1133 =====
1134
1135 After replacing an old implementation:
1136
1137 remove unused:
1138
1139 JS
1140 CSS
1141 HTML
1142 functions
1143 API routes
1144 database fields
1145
1146
1147 Do not leave five versions of the same feature.
1148
1149
1150 =====

1151 49. NO HARDCODED BUSINESS VALUES

1152 =====

1153

1154 Avoid putting important business rules directly into HTML.

1155

1156 For example:

1157

1158 BAD:

1159

1160 `if price > 10000000`

1161

1162

1163 Better:

1164

1165 business configuration

1166 +

1167 backend rules

1168

1169

1170 This allows rules to change without rewriting the frontend.

1171

1172

1173 =====

1174 50. GHAR MASTER RULE

1175 =====

1176

1177 EVERY NEW FEATURE MUST ANSWER:

1178

1179 1. What problem does it solve?

1180 2. Who can use it?

1181 3. What data does it need?

1182 4. What database tables are involved?

1183 5. What API endpoints are required?

1184 6. What backend service is required?

1185 7. What validation is required?

1186 8. What frontend JS is required?

1187 9. What HTML is required?

1188 10. What CSS is required?

1189 11. What happens `if` it fails?

1190 12. What security risks exist?

1191 13. How will it be tested?

1192

1193

1194 =====

1195 51. FINAL DEVELOPMENT FORMULA

1196 =====

```
1197
1198 REQUIREMENT
1199     ↓
1200 ARCHITECTURE
1201     ↓
1202 DATA MODEL
1203     ↓
1204 DATABASE
1205     ↓
1206 MIGRATION
1207     ↓
1208 BACKEND SERVICE
1209     ↓
1210 API
1211     ↓
1212 VALIDATION
1213     ↓
1214 JAVASCRIPT
1215     ↓
1216 HTML
1217     ↓
1218 CSS
1219     ↓
1220 TEST
1221     ↓
1222 SECURITY CHECK
1223     ↓
1224 GIT COMMIT
1225     ↓
1226 BACKUP
1227     ↓
1228 NEXT FEATURE
1229
1230
1231 =====
1232 52. GHAR GOLDEN RULE
1233 =====
1234
1235 DO NOT ASK:
1236
1237 "Where can I add this code?"
1238
1239 ASK:
1240
1241 "Where does this feature belong in the architecture?"
1242
```

1243
1244 Then place it in the correct layer.
1245
1246
1247 =====
1248 53. THE MOST IMPORTANT DOCUMENT
1249 =====
1250
1251 Create:
1252
1253 GHAR_MASTER_REGISTRY.md
1254
1255
1256 It should contain:
1257
1258 1. Complete folder structure
1259 2. All HTML files
1260 3. All CSS files
1261 4. All JS files
1262 5. All JSON files
1263 6. All database tables
1264 7. All database relationships
1265 8. All API endpoints
1266 9. All user roles
1267 10. All permissions
1268 11. All status values
1269 12. All IDs
1270 13. All data-* attributes
1271 14. All environment variables
1272 15. All external services
1273 16. All important business rules
1274 17. All test requirements
1275
1276
1277 This becomes the single reference document for the entire
1278 GHAR project.
1279
1280
1281 =====
1282 54. FINAL SAFETY CHECK BEFORE PRODUCTION
1283 =====
1284
1285 [] No broken links
1286 [] No broken imports
1287 [] No duplicate declarations
1288 [] No missing API endpoints

1289 [] No missing database tables
1290 [] No unauthorized API access
1291 [] No exposed secrets
1292 [] No public private documents
1293 [] No payment trust on frontend
1294 [] No broken mobile layout
1295 [] No critical console errors
1296 [] No critical server errors
1297 [] Database backup available
1298 [] Rollback strategy available
1299 [] Monitoring available
1300 [] Error logging available
1301 [] SSL enabled
1302 [] Production environment separated
1303 [] Security review completed
1304 [] Full user journey tested

1305

1306

1307 =====

1308 END OF GHAR MASTER RULES

1309 =====